Lerneinheit 4 · Projektaufgabe: Der Datenworkflow am zweiten Datensatz

M2 · Data-Science-Werkzeugkasten

Version 1.0

Lernziele

Nach dieser Projektaufgabe können Sie …

- ☐ einen unbekannten Datensatz einlesen, seine Spalten nach Rolle einordnen und begründen, welche davon für eine Fragestellung gebraucht werden,
- ☐ eine Ladefunktion mit Fehlervertrag schreiben, die eine fehlende Datei durchlässt und fehlende Spalten mit verständlicher Meldung abweist,
- ☐ entscheiden, welche fehlenden Werte eine Beobachtung unbrauchbar machen, den Vertrag einer bereinigten Tabelle selbst formulieren und eine Bereinigungsfunktion dagegen schreiben und prüfen,
- ☐ Fragen an die Daten mit `groupby`, `unstack` und `crosstab` beantworten und die Wirkung fehlender Werte auf Kennzahlen benennen,
- ☐ ein Analyseblatt mit Balken-, Histogramm- und Streudiagramm zeichnen und daraus ablesen, welche Klassen sich trennen lassen,
- ☐ die Maße als Array übergeben und darauf mit Achsen-Reduktionen, Broadcasting und Masken rechnen: Spalten standardisieren, Mittelwerte je Art bilden und das Ergebnis mit pandas abgleichen,
- ☐ die bereinigte Tabelle als Übergabe speichern und zurücklesen.

Im Termin zu LE04 haben Sie den Datenworkflow an der Passagierliste der Titanic durchlaufen: einlesen, prüfen, nach einem Vertrag bereinigen, gruppieren, zeichnen, als Array übergeben. In dieser Projektaufgabe durchlaufen Sie denselben Weg allein, an einem zweiten Datensatz und mit einem Unterschied: Bei Titanic waren die Probleme eingebaut und der Vertrag vorgegeben. Hier sind die Probleme echt, und den Vertrag schreiben Sie selbst.

Der Auftrag: Eine Forschungsgruppe hat drei Saisons lang Pinguine auf drei Inseln der Antarktis vermessen. Sie will ihre Feldbeobachtungen auswerten und später die Art eines Pinguins aus seinen Maßen bestimmen. Sie bekommen die Rohtabelle der Studie, so wie sie veröffentlicht ist, und liefern eine bereinigte Tabelle mit nachvollziehbaren Entscheidungen, drei Auswertungen, ein Analyseblatt, die Maße als standardisiertes Array mit einer ersten Vorhersage der Art und die Übergabe an die nächste Lerneinheit, in der auf dieser Tabelle ein Modell entsteht. pandas trägt den Weg bis zur Tabelle, NumPy übernimmt, sobald gerechnet wird.

Planen Sie rund drei Stunden ein: etwa 165 Minuten für die sieben Pflichtschritte und den Verständnisnachweis am Ende, der Rest ist Reserve. Die Minutenangaben an den Aufgaben sind eine Orientierung, keine Vorgabe. Jeder Schritt nennt in seiner ersten Zeile, welches Kapitel er voraussetzt; Grundkurs Teil 3 sollten Sie vorher durchgearbeitet haben. Unter jeder Aufgabe liegt die Lösung zum Aufklappen, und an jedem Schritt gibt es eine Anschlusslösung, falls Sie nicht weiterkommen. An mehreren Stellen gibt es mehr als eine richtige Entscheidung; die Prüfzellen prüfen Ihre Tabelle gegen Ihren eigenen Vertrag und gegen Regeln, die für jede vertretbare Entscheidung gelten, nicht gegen eine Musterlösung. Die Lösung zeigt eine Fassung mit Begründung.

Beim Bearbeiten entsteht neben dem Notebook ein Ordner `out/` für Dateien, die Sie schreiben. Die Rohdaten in `data/` werden nie verändert.

Drei Leitfragen

Alle Lernziele dieser Aufgabe lassen sich auf drei Fragen zurückführen. Stellen Sie sie sich an jedem Schritt, und beantworten Sie sie am Ende im Verständnisnachweis.

1. **Kann ich begründen, welche Daten ich behalte und welche ich verändere?** (Vertrag und Bereinigung)
2. **Kann ich eine fachliche Frage in Tabellen- und Array-Operationen übersetzen?** (Fragen, Analyseblatt, Array)
3. **Kann ich erklären, ob mein Ergebnis zur Frage passt, und wo seine Aussage endet?** (Denkfragen, Diagramme, Stichprobe)

Inhalt

- 4.1 Der Datensatz
- 4.2 Laden mit Fehlervertrag
- 4.3 Vertrag und Bereinigung
- 4.4 Merkmale ableiten
- 4.5 Fragen an die Daten
- 4.6 Analyseblatt
- 4.7 Von der Tabelle zum Array
- 4.8 Übergabe

4.1 Der Datensatz

Setzt voraus: LE04, Kapitel LE 4, Abschnitt 4.2.

Die Datei `penguins_raw.csv` ist die Rohdatentabelle der Studie von Gorman, Williams und Fraser (Palmer Station, 2007 bis 2009), unverändert aus dem R-Paket palmerpenguins übernommen, Lizenz CC0. Das Datenblatt `DATENBLATT_penguins.md` neben der Datei beschreibt jede Spalte. Die nächste Zelle stellt Daten und Bibliotheken bereit, die Zelle danach zeigt, was in der Datei steckt.



Die Entscheidungen stehen im Vertrag (CONTRACT), die Prüfzellen prüfen gegen ihn. N ist 342 oder 333, je nach Entscheidung zu den Zeilen ohne Geschlecht

```
# Vorgabe: Kursdaten bereitstellen – einfach ausführen
# Die Zelle legt den Ordner data/ an und lädt fehlende Dateien aus dem Kursrepo nach:
#   data/penguins_raw.csv
#   data/DATENBLATT_penguins.md
```

```
# Vorgabe: Bibliotheken und ein erster Blick – einfach ausführen
import json, os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

PANDAS_3 = int(pd.__version__.split(".")[0]) >= 3
if not PANDAS_3:
    pd.options.mode.copy_on_write = True
    print(f"pandas {pd.__version__}: Copy-on-Write eingeschaltet; Textspalten erscheinen hier als object statt str")
else:
    print("pandas", pd.__version__)
os.makedirs("out", exist_ok=True)

raw = pd.read_csv("data/penguins_raw.csv", encoding="utf-8")
print(raw.shape)
print(pd.concat([raw.dtypes.rename("dtype"), raw.isna().sum().rename("fehlend")], axis=1))
```

① `isna().sum()` je Spalte, daneben der Datentyp: zwei Spalten in einer Tabelle, mit `concat` entlang Achse 1

```
pandas 3.0.5
(344, 17)
                     dtype  fehlend
studyName              str        0
Sample Number        int64        0
Species                str        0
Region                 str        0
Island                 str        0
Stage                  str        0
Individual ID          str        0
Clutch Completion      str        0
Date Egg               str        0
Culmen Length (mm)   float64        2
Culmen Depth (mm)    float64        2
Flipper Length (mm)  float64        2
Body Mass (g)        float64        2
Sex                    str       11
Delta 15 N (o/oo)    float64       14
Delta 13 C (o/oo)    float64       13
Comments               str      290
```

```
print(raw.iloc[[0, 3]].T.to_string())     # zwei Tiere, transponiert wie in NumPy: je Spalte eine Zeile
```

```
                                                    0                                  3
studyName                                      PAL0708                            PAL0708
Sample Number                                        1                                  4
Species              Adelie Penguin (Pygoscelis adeliae)  Adelie Penguin (Pygoscelis adeliae)
Region                                           Anvers                             Anvers
Island                                         Torgersen                          Torgersen
Stage                                Adult, 1 Egg Stage                 Adult, 1 Egg Stage
Individual ID                                       N1A1                               N2A2
Clutch Completion                                    Yes                                Yes
Date Egg                                      2007-11-11                         2007-11-16
Culmen Length (mm)                                  39.1                                NaN
Culmen Depth (mm)                                   18.7                                NaN
Flipper Length (mm)                                181.0                                NaN
Body Mass (g)                                     3750.0                                NaN
Sex                                                 MALE                                NaN
Delta 15 N (o/oo)                                    NaN                                NaN
Delta 13 C (o/oo)                                    NaN                                NaN
Comments                 Not enough blood for isotopes.                Adult not sampled.
```

Links ein vollständig vermessenes Tier, rechts das erste der beiden ohne Maße; sein Kommentar erklärt die Lücken.

Siebzehn Spalten, und nicht alle davon sind Merkmale eines Pinguins. Bevor Sie etwas bereinigen, ordnen Sie jede Spalte einer Rolle zu. Das ist die erste Entscheidung des Workflows, und sie entscheidet später, welche Spalten in die bereinigte Tabelle kommen.

Überblick: Rollen einer Spalte

Rolle	Bedeutung	Beispiel aus Titanic
`measure`	gemessene Zahl, ein Merkmal für das Modell	`age`, `fare`
`category`	Kategorie, ein Merkmal für das Modell nach Kodierung	`sex`, `embarked`
`target`	die Zielgröße, die das Modell später vorhersagen soll	`survived`
`id`	Kennung der Beobachtung oder der Erhebung, kein Merkmal	`ticket`
`comment`	Freitext, kein Merkmal	`name`
`constant`	in allen Zeilen derselbe Wert, trägt keine Information	
`date`	Datum als Text	

✏️ Aufgabe 4.1 #1 5 min

Füllen Sie das Dict `roles` aus: ein Eintrag je Spalte der Rohtabelle, der Wert ist eine der sieben Rollen aus der Übersicht als Text. Die Prüfzelle prüft, ob jede Spalte eine gültige Rolle hat, und rechnet bei den konstanten Spalten nach. Bei `studyName` sind `id` und `date` beide richtig, denn der Name enthält die Saison.

Hinweis: `raw[col].nunique()` zählt die verschiedenen Werte einer Spalte; eine Spalte mit dem Ergebnis 1 ist konstant. Die Isotopenwerte sind Messwerte, auch wenn sie in dieser Aufgabe später nicht gebraucht werden.

→ Lösung im Anhang

```
roles = {
    # TODO: Ihre Lösung hier
    ...
}
```

```
# Prüfzelle – einfach ausführen
ROLES = {"measure", "category", "target", "id", "comment", "constant", "date"}
assert isinstance(roles, dict) and roles, "roles ist noch leer: ein Eintrag je Spalte der Rohtabelle"
assert set(roles) == set(raw.columns), f"es fehlen Spalten oder es sind zu viele: {set(roles) ^
set(raw.columns)}"
assert all(r in ROLES for r in roles.values()), f"unbekannte Rolle: {set(roles.values()) - ROLES}"
constant = {c for c in raw.columns if raw[c].nunique() == 1}
assert {c for c, r in roles.items() if r == "constant"} == constant, f"konstant sind genau
{sorted(constant)}"
assert roles["Species"] == "target", "Species ist die Zielgröße: die Art soll später vorhergesagt werden"
for col in ["Culmen Length (mm)", "Culmen Depth (mm)", "Flipper Length (mm)", "Body Mass (g)"]:
    assert roles[col] == "measure", f"{col} ist ein Messwert"
assert roles["Comments"] == "comment", "Comments ist Freitext"
assert roles["Individual ID"] == "id" and roles["Sample Number"] == "id", "Individual ID und Sample Number
sind Kennungen"
assert roles["studyName"] in {"id", "date"}, "studyName ist Kennung der Saison (oder ein Datum)"
assert roles["Island"] == "category" and roles["Sex"] == "category", "Island und Sex sind Kategorien"
print("Alle Prüfungen bestanden ✓")
```

💭 Kurz überlegen

`Individual ID` hat 190 verschiedene Werte für 344 Zeilen. Ist es dann eine Kennung?

```
print("Individual ID allein eindeutig:", raw["Individual ID"].is_unique)
print("mit studyName zusammen eindeutig:", not raw.duplicated(subset=["studyName", "Individual ID"]).any())
print(raw.loc[raw["Individual ID"] == "N1A1", ["studyName", "Species", "Sex"]].to_string(index=False))
```

```
Individual ID allein eindeutig: False
mit studyName zusammen eindeutig: True
studyName                                Species    Sex
  PAL0708 Adelie Penguin (Pygoscelis adeliae)   MALE
  PAL0910   Gentoo penguin (Pygoscelis papua) FEMALE
```

4.2 Laden mit Fehlervertrag

Setzt voraus: Grundkurs Teil 3, Kapitel LE 3, Abschnitt 3.1 und LE 3, Abschnitt 3.2; LE04, Kapitel LE 4, Abschnitt 4.2.

Die Spaltennamen der Rohdatei enthalten Leerzeichen, Klammern und Einheiten. Für die Arbeit in pandas bekommen sie kurze Namen ohne Leerzeichen; die Einheit bleibt im Namen, damit niemand Gramm für Kilogramm hält. Das Dict `RENAME` legt die Zuordnung fest, `df.rename(columns=RENAME)` wendet sie an und liefert eine neue Tabelle.

```
# Vorgabe: die neuen Spaltennamen
RENAME = {
    "studyName": "study", "Sample Number": "sample_number", "Species": "species", "Region": "region",
    "Island": "island", "Stage": "stage", "Individual ID": "individual_id",
    "Clutch Completion": "clutch_completion", "Date Egg": "date_egg",
    "Culmen Length (mm)": "culmen_length_mm", "Culmen Depth (mm)": "culmen_depth_mm",
    "Flipper Length (mm)": "flipper_length_mm", "Body Mass (g)": "body_mass_g", "Sex": "sex",
    "Delta 15 N (o/oo)": "delta_15n", "Delta 13 C (o/oo)": "delta_13c", "Comments": "comments",
}
print(len(RENAME), "Spalten; Beispiel:", list(RENAME.items())[9])
```

```
17 Spalten; Beispiel: ('Culmen Length (mm)', 'culmen_length_mm')
```

Definition: Vertrag von load_penguins

`load_penguins(path)` liest die Datei unter `path` mit `pd.read_csv(path, encoding="utf-8")`, benennt die Spalten mit `RENAME` um und gibt den DataFrame mit 17 Spalten zurück, Zeilen und Werte unverändert.

- Fehlt die Datei, läuft der `FileNotFoundError` aus `read_csv` durch. Die Funktion fängt ihn nicht ab, denn sie kann ihn nicht beheben, und die Meldung von `read_csv` nennt den Pfad.
- Fehlt eine der 17 Spalten in der Datei, löst die Funktion einen `ValueError` aus, dessen Meldung die fehlenden Spalten nennt. Eine Datei mit anderen Spalten würde sonst still falsche Ergebnisse liefern.

✏️ Aufgabe 4.2 #1 15 min

Schreiben Sie `load_penguins(path)` nach dem Vertrag. Die Prüfzelle prüft den Normalfall, eine kleine zweite Datei mit drei Zeilen und beide Fehlerfälle.

Hinweis: `set(RENAME) - set(df.columns)` ist die Menge der Spalten, die fehlen; ist sie nicht leer, `raise ValueError(f"missing columns in {path}: {sorted(missing)}")`. Die Meldungen in `raise` sind im Kurs englisch, wie der Code.

→ Lösung im Anhang

```
def load_penguins(path):
    """Liest die Rohtabelle der Palmer-Pinguine und benennt die Spalten um."""
    # TODO: Ihre Lösung hier
    ...
```

```
    # Prüfzelle – einfach ausführen (legt eigene Dateien in out/ an)
    df = load_penguins("data/penguins_raw.csv")
    assert df is not None, "load_penguins gibt noch nichts zurück (return vergessen?)"
    assert df.shape == (344, 17), f"(344, 17) erwartet, nicht {df.shape}"
    assert list(df.columns) == list(RENAME.values()), "die Spalten sollen die Namen aus RENAME tragen, in der
Reihenfolge der Datei"
    assert df["culmen_length_mm"].dtype == "float64" and df["culmen_length_mm"].isna().sum() == 2,
"culmen_length_mm: float64 mit zwei Lücken"
    assert df["sex"].isna().sum() == 11, "sex hat in der Rohdatei 11 Lücken"
    raw.head(3).to_csv("out/three.csv", index=False)                    # zweite Datei: die Funktion muss
path lesen
    tiny = load_penguins("out/three.csv")
    assert tiny.shape == (3, 17) and tiny["island"].tolist() == ["Torgersen"] * 3, "die kleine Datei soll drei
Zeilen liefern"
    try:
        load_penguins("data/gibt_es_nicht.csv")
        raise AssertionError("eine fehlende Datei soll einen FileNotFoundError auslösen")
    except FileNotFoundError:
        pass
    raw.drop(columns=["Sex"]).head(3).to_csv("out/no_sex.csv", index=False)
    try:
        load_penguins("out/no_sex.csv")
        raise AssertionError("eine Datei ohne die Spalte Sex soll einen ValueError auslösen")
    except ValueError as e:
        assert "Sex" in str(e), f"die Meldung soll die fehlende Spalte nennen, lautet aber: {e}"
    print("Alle Prüfungen bestanden ✓")
```

4.3 Vertrag und Bereinigung

Setzt voraus: LE04, Kapitel LE 4, Abschnitt 4.4.

Zuerst die Falle, in die jeder einmal läuft. `dropna()` ohne Argument entfernt jede Zeile, in der irgendein Wert fehlt. Was das hier bedeutet, zeigt die nächste Zelle.

```
    print("Zeilen:", len(df), "| nach dropna():", len(df.dropna()))
    print(df.isna().sum()[lambda s: s > 0].to_dict())
```

```
Zeilen: 344 | nach dropna(): 34
{'culmen_length_mm': 2, 'culmen_depth_mm': 2, 'flipper_length_mm': 2, 'body_mass_g': 2, 'sex': 11,
'delta_15n': 14, 'delta_13c': 13, 'comments': 290}
```

> 💭 **Kurz überlegen**
>
> Welche Lücken machen eine Zeile für den Auftrag unbrauchbar, und welche nicht?

Entscheidungsfelder. Den Vertrag der bereinigten Tabelle schreiben Sie zum Teil selbst. Einiges ist festgelegt, weil es Kursregel ist oder weil sonst alles danach umständlich wird; anderes entscheiden Sie. Jede Entscheidung halten Sie in drei Teilen fest: **Entscheidung, Begründung, Folge für die Auswertung.** Zum Beispiel: „Ich behalte die Tiere ohne Geschlechtsangabe. Ihre Maße sind vollständig und bleiben für die Auswertung verfügbar. Vergleiche nach Geschlecht verwenden neun Tiere weniger." Beide Antworten sind in diesen Feldern richtig, wenn Begründung und Folge stimmen.

Überblick: Was feststeht und was Sie entscheiden

Feld	Festgelegt oder frei	Rahmen
Spaltenauswahl	frei, mit Grenzen	`species`, die vier Maße, `island` und `sex` bleiben. Kennungen, konstante Spalten, Kommentare und Isotope kommen nicht in die bereinigte Tabelle
Spaltennamen	frei für zusätzliche Spalten	die Namen aus `RENAME` für `species`, `island`, `sex` und die vier Maße sind verbindlich, die Prüfzellen brauchen sie
Zeilen ohne Maße	fest	die zwei Zeilen entfallen; die Maßspalten haben keine Lücken
Zeilen ohne Geschlecht	frei	behalten (342 Zeilen, `sex` erlaubt Lücken) oder entfernen (333 Zeilen)
`clutch_completion`	frei	als `bool`, als Text oder weglassen; wer umwandelt, prüft vorher, dass nur `Yes` und `No` vorkommen
Jahr	frei, Zusatzaufgabe	aus `study` oder `date_egg` ableiten, vor der Spaltenauswahl
Kursregeln	fest	Lücken werden nicht gefüllt, kein Wert wird erfunden, die Eingabe bleibt unverändert, keine doppelten Zeilen, Index von 0 bis N minus 1

✏️ Aufgabe 4.3 #1 10 min

Der Vertrag ist ein Dict `CONTRACT`. Die sechs festgelegten Spalten sind schon eingetragen: je Spalte ein Eintrag mit `kind` (`"text"`, `"bool"`, `"int"` oder `"float"`) und entweder `values` (Menge der zulässigen Werte) oder `range` (Tupel aus Minimum und Maximum, das die Werte einschließen darf); `"nullable": True` steht dort, wo Lücken erlaubt sind, sonst entfällt der Schlüssel. Lesen Sie die sechs Zeilen, dann treffen Sie Ihre zwei Entscheidungen und ergänzen `sex` (mit oder ohne `nullable`) und `clutch_completion` (als `bool`, als Text oder gar nicht). Dazu unter `rows` in einem Satz, welche Zeilen entfallen, und unter `decisions` je Entscheidungsfeld die drei Teile Entscheidung, Begründung, Folge. Die Lösung zeigt eine Fassung, die die neun Zeilen ohne Geschlecht behält; die Alternative ist ebenso richtig.

Hinweis: Die Wertebereiche der Maße sind großzügig gerundet; ein Vertrag beschreibt, was plausibel ist, nicht die zufälligen Grenzen dieser Stichprobe (`df[cols].agg(["min", "max"])` zeigt sie). Ein Eintrag für `sex` sieht aus wie der für `island`, nur mit den Werten `female` und `male`.

→ *Lösung im Anhang*

```
CONTRACT = {
    "columns": {
        "species": {"kind": "text", "values": {"Adelie", "Chinstrap", "Gentoo"}},   # fest
        "island": {"kind": "text", "values": {"Biscoe", "Dream", "Torgersen"}},   # fest
        "culmen_length_mm": {"kind": "float", "range": (30.0, 65.0)},   # fest
        "culmen_depth_mm": {"kind": "float", "range": (12.0, 25.0)},    # fest
        "flipper_length_mm": {"kind": "int", "range": (170, 240)},      # fest
        "body_mass_g": {"kind": "float", "range": (2500.0, 6500.0)},    # fest
        # Ihre Entscheidungen: sex (nullable?) und clutch_completion (bool, Text oder weglassen)
        # TODO: Ihre Lösung hier
        ...
    },
    "rows": "...",                       # ein Satz: welche Zeilen entfallen
    "decisions": {                      # je Feld: Entscheidung. Begründung. Folge für die Auswertung.
        # TODO: Ihre Lösung hier
        ...
    },
}
# TODO: Ihre Lösung hier
...
```

Die Vertragsprüfung ist eine Vorgabe. `check_contract(df, contract)` prüft eine Tabelle gegen genau so ein Dict: die Spaltenmenge, die Art der Werte (geprüft an den Werten selbst, nicht am Namen des Datentyps), die zulässigen Werte oder den Bereich und ob Lücken nur dort stehen, wo der Vertrag sie mit `nullable` erlaubt. Lesen Sie sie einmal, sie ist das Gegenstück zur Prüfzelle von `clean_titanic` im Termin, nur mit dem Vertrag als Daten statt als Code.

```
# Vorgabe: Vertragsprüfung — einfach ausführen
def check_contract(df, contract):
    cols = contract["columns"]
    assert set(df.columns) == set(cols), f"Spalten passen nicht zum Vertrag: {set(df.columns) ^ set(cols)}"
    for col, rule in cols.items():
        s = df[col]
        vals = s.dropna()
①       if rule["kind"] == "text":
            assert all(isinstance(v, str) for v in vals), f"{col}: Text erwartet"
        elif rule["kind"] == "bool":
            assert all(isinstance(v, (bool, np.bool_)) for v in vals), f"{col}: True/False erwartet"
        elif rule["kind"] == "int":
            assert s.dtype.kind == "i", f"{col}: ganzzahliger Datentyp erwartet, ist {s.dtype}"
        elif rule["kind"] == "float":
            assert s.dtype.kind == "f", f"{col}: Fließkomma-Datentyp erwartet, ist {s.dtype}"
        else:
            raise ValueError(f"unknown kind {rule['kind']!r} for column {col!r}")
        if "values" in rule:
            assert set(vals) <= set(rule["values"]), f"{col}: unzulässige Werte {set(vals) -
set(rule['values'])}"
        if "range" in rule:
            lo, hi = rule["range"]
            assert vals.between(lo, hi).all(), f"{col}: Werte außerhalb von {lo} bis {hi}"
        if not rule.get("nullable", False):
            assert s.notna().all(), f"{col}: Lücken sind laut Vertrag nicht erlaubt, es gibt
{s.isna().sum()}"
    return True

print("check_contract bereit")
```

① die Art einer Spalte wird an den Werten geprüft: so gilt die Prüfung unter pandas 3 (`str`) und älteren Versionen (`object`)

✏️ Aufgabe 4.3 #2 30 min

Schreiben Sie `clean_penguins(df)`: Sie bekommt die Tabelle aus `load_penguins` und gibt eine neue Tabelle zurück, die `CONTRACT` erfüllt und die Kursregeln einhält. Die Prüfzelle prüft zuerst Ihren Vertrag und danach die Regeln, die für jede Entscheidung gelten.

Zusatzaufgabe: Leiten Sie vor der Spaltenauswahl das Jahr ab (`study` ist `PAL0708` für die Saison 2007/08: `df["study"].str[3:5].astype(int) + 2000`) und nehmen Sie es als Spalte `year` in eine zweite Fassung von Funktion und Vertrag auf. Zählen Sie die Tiere je Jahr.

Fachlicher Hinweis (erst das): Sechs Teilschritte, in der Reihenfolge aus dem Termin: die Spalten des Vertrags auswählen; die Zeilen ohne Maße entfernen (und nach Ihrer Entscheidung die ohne Geschlecht); die Art auf den Kurznamen bringen; das Geschlecht kleinschreiben; `clutch_completion` nach Ihrer Entscheidung umwandeln; die Flossenlänge ganzzahlig machen; zuletzt Duplikate entfernen und den Index neu setzen. Schreiben Sie die Schritte als Kommentarzeilen in die Funktion, bevor Sie Code schreiben.

Technischer Hinweis (wenn ein Schritt hakt): `dropna(subset=[...])`; `.str.split(" ").str[0]` nimmt das erste Wort und umgeht die uneinheitliche Großschreibung von „Penguin“; `.str.lower()`; vor `== "Yes"` mit `set(...)` prüfen, dass nur `Yes` und `No` vorkommen; `.astype("int64")` geht erst, wenn die Spalte keine Lücken mehr hat; `drop_duplicates().reset_index(drop=True)`. Wenn es dann noch nicht läuft: die Anschlusslösung unter der Prüfzelle.

→ *Lösung im Anhang*

```python
def clean_penguins(df):
    """Bereinigt die Rohtabelle nach CONTRACT und gibt eine neue Tabelle zurück."""
    # TODO: Ihre Lösung hier
    ...
```

```
   # Prüfzelle – einfach ausführen
   df = load_penguins("data/penguins_raw.csv")                    # frischer Stand
   before = df.copy()
   clean = clean_penguins(df)
   assert clean is not None, "clean_penguins gibt noch nichts zurück (return vergessen?)"
① assert isinstance(CONTRACT["columns"], dict) and CONTRACT["columns"], "CONTRACT['columns'] ist noch leer: ein Eintrag je Spalte der bereinigten Tabelle"
   assert CONTRACT["rows"] != "..." and CONTRACT["decisions"], "CONTRACT: rows als Satz und decisions ausfüllen"
   assert {"Zeilen ohne Geschlecht", "clutch_completion"} <= set(CONTRACT["decisions"]), "decisions: je ein Eintrag für die Zeilen ohne Geschlecht und für clutch_completion"
   assert all(len(v.split(".")) >= 3 for v in CONTRACT["decisions"].values()), "decisions: je Feld drei Teile als Sätze: Entscheidung. Begründung. Folge."
   check_contract(clean, CONTRACT)
② assert df.equals(before), "die Eingabe wurde verändert; clean_penguins soll eine neue Tabelle liefern"
   MEASURES = ["culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"]
   for col in ["species", "island", "sex"] + MEASURES:
       assert col in clean.columns, f"{col} muss in der bereinigten Tabelle bleiben"
   for col in ["study", "sample_number", "individual_id", "region", "stage", "comments", "delta_15n", "delta_13c"]:
       assert col not in clean.columns, f"{col} gehört nicht in die bereinigte Tabelle (Kennung, Konstante, Kommentar oder Isotop)"
   assert clean[MEASURES].notna().all().all(), "die Maßspalten dürfen keine Lücken haben (die zwei Zeilen ohne Maße entfallen)"
   assert len(clean) in (333, 342), f"342 Zeilen (Geschlecht fehlend behalten) oder 333 (entfernt) erwartet, nicht {len(clean)}"
   assert set(clean["species"]) == {"Adelie", "Chinstrap", "Gentoo"}, f"species: Kurznamen Adelie, Chinstrap, Gentoo erwartet, nicht {set(clean['species'])}"
   assert set(clean["sex"].dropna()) == {"female", "male"}, f"sex: female und male in Kleinbuchstaben erwartet, nicht {set(clean['sex'].dropna())}"
   assert all(clean[c].nunique(dropna=False) > 1 for c in clean.columns), "eine Spalte mit nur einem Wert trägt keine Information"
   assert clean.duplicated().sum() == 0, "keine doppelten Zeilen"
   assert list(clean.index) == list(range(len(clean))), "Index von 0 bis N-1 (reset_index)"
③ raw_keys = set(zip(*(df[c] for c in MEASURES)))
   assert all(k in raw_keys for k in zip(*(clean[c] for c in MEASURES))), "Maße wurden verändert oder gefüllt; sie müssen aus der Rohtabelle stammen"
   assert len(clean) == len(df.dropna(subset=MEASURES)) - (0 if clean["sex"].isna().any() else df.dropna(subset=MEASURES)["sex"].isna().sum()), "es fehlen Zeilen, die nach Ihrer Regel bleiben müssten"
   print(f"Ihre Tabelle: {clean.shape}, {clean['sex'].isna().sum()} Zeilen ohne Geschlecht")
   print("Alle Prüfungen bestanden ✓")
```

① zuerst der eigene Vertrag
② dann die Regeln, die für jede Entscheidung gelten
③ nichts gefüllt, nichts erfunden: jede Maßkombination der Ausgabe kommt in der Eingabe vor

```
   # Lösung der Zusatzaufgabe (Jahr) in einer eigenen Zelle: zweite Fassung von Funktion und Vertrag
   # TODO: Ihre Lösung hier
   ...
```

4.4 Merkmale ableiten

Setzt voraus: LE04, Kapitel LE 4, Abschnitt 4.4 (Zuweisen).

Zwei Spalten, die der Auftrag braucht, stehen nicht in der Datei, lassen sich aber aus ihr ableiten: die Körpermasse in Kilogramm, wie die Biologen sie angeben, und das Verhältnis von Schnabellänge zu Schnabeltiefe, eine Zahl ohne Einheit, die die Form des Schnabels beschreibt. Beides kommt als neue Spalte dazu, die Rohspalten bleiben; so ist die Ableitung sichtbar und umkehrbar. `df["neu"] = df["alt"] / 1000` ordnet über denselben Index zu, hier also Zeile für Zeile.

✏️ Aufgabe 4.4 #1 8 min

Schreiben Sie `add_features(df)`: Sie gibt eine neue Tabelle mit den Spalten `body_mass_kg` (Masse in Kilogramm) und `culmen_ratio` (Länge durch Tiefe) zusätzlich zu allen vorhandenen zurück. Die Eingabe bleibt unverändert.

Hinweis: Erst `out = df.copy()`, dann die zwei Zuweisungen, dann `return out`. Ohne die Kopie würden die Zuweisungen in `df` schreiben.

→ Lösung im Anhang

```
def add_features(df):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
before = clean.copy()
feat = add_features(clean)
assert feat is not None, "add_features gibt noch nichts zurück (return vergessen?)"
assert clean.equals(before), "die Eingabe wurde verändert (df.copy() vergessen?)"
assert feat.shape == (len(clean), clean.shape[1] + 2), f"zwei Spalten mehr erwartet, nicht {feat.shape}"
assert np.allclose(feat["body_mass_kg"], clean["body_mass_g"] / 1000), "body_mass_kg ist body_mass_g / 1000"
assert np.allclose(feat["culmen_ratio"], clean["culmen_length_mm"] / clean["culmen_depth_mm"]), "culmen_ratio ist Länge / Tiefe"
assert feat["culmen_ratio"].between(1.5, 3.7).all(), "culmen_ratio liegt zwischen rund 1.6 und 3.6"
print(feat.groupby("species")[["body_mass_kg", "culmen_ratio"]].mean().round(3))
print("Alle Prüfungen bestanden ✓")
```

4.5 Fragen an die Daten

Setzt voraus: LE04, Kapitel LE 4, Abschnitt 4.5.

Drei Fragen der Forschungsgruppe, jede eine Funktion und jede ein `groupby` oder ein `crosstab`. Die Fragen sind bewusst als Fragen an diesen Datensatz formuliert, nicht an alle Pinguine der Antarktis: Eine Tabelle beantwortet, was in ihr steht. Die Prüfzellen rechnen die Erwartung aus Ihrer Tabelle `clean` auf einem zweiten Weg nach; wer im Vertrag anders entschieden hat, bekommt andere Zahlen und trotzdem grüne Prüfzellen.

💭 **Kurz überlegen**

Bevor Sie die Tabelle bauen: Welche Form hat sie, und wie viele ihrer Einträge erwarten Sie als Null?

✏️ Aufgabe 4.5 #1 8 min

Welche Arten wurden in diesem Datensatz auf welchen Inseln beobachtet? Schreiben Sie `counts_by_island(df)`: eine Tabelle mit den Arten in den Zeilen, den Inseln in den Spalten und der Anzahl der Tiere als Werte, Nullen statt Lücken.

Hinweis: `pd.crosstab(df["species"], df["island"])` oder `df.groupby(["species", "island"]).size().unstack(fill_value=0)`. Beides liefert dieselbe Tabelle.

→ Lösung im Anhang

```
def counts_by_island(df):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
table = counts_by_island(clean)
assert table is not None, "counts_by_island gibt noch nichts zurück"
assert table.shape == (3, 3) and list(table.index) == ["Adelie", "Chinstrap", "Gentoo"], "drei Arten in den Zeilen, drei Inseln in den Spalten"
for sp in table.index:                                  # Referenz über Masken, unabhängig von groupby
    for isl in table.columns:
        expected = int(((clean["species"] == sp) & (clean["island"] == isl)).sum())
        assert table.loc[sp, isl] == expected, f"{sp} auf {isl}: {expected} erwartet, nicht {table.loc[sp, isl]}"
assert table.to_numpy().sum() == len(clean), "jedes Tier zählt genau einmal"
assert (table.loc["Chinstrap"] > 0).sum() == 1 and (table.loc["Gentoo"] > 0).sum() == 1, "Chinstrap und Gentoo leben je auf genau einer Insel"
print(table)
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 4.5 #2

10 min

Wie schwer sind die vermessenen Tiere je Art und Geschlecht, in Kilogramm? Schreiben Sie `mass_by_species_sex(df)`: mittlere Körpermasse aus der Spalte `body_mass_kg`, Arten in den Zeilen, Geschlechter in den Spalten. Die Funktion bekommt `feat` aus dem letzten Kapitel. Tiere ohne Geschlecht fallen dabei heraus, ohne dass Sie etwas tun müssen: `groupby` lässt fehlende Schlüssel weg.

Hinweis: `groupby` mit zwei Schlüsseln, `["body_mass_kg"].mean()`, dann `unstack()`.

→ Lösung im Anhang

```
def mass_by_species_sex(df):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
table = mass_by_species_sex(feat)
assert table is not None, "mass_by_species_sex gibt noch nichts zurück"
assert table.shape == (3, 2) and list(table.columns) == ["female", "male"], "Arten in den Zeilen, female und male in den Spalten"
for sp in table.index:
    for sx in table.columns:
        mask = (clean["species"] == sp) & (clean["sex"] == sx)
        assert abs(table.loc[sp, sx] - clean.loc[mask, "body_mass_g"].mean() / 1000) < 1e-6, f"{sp}/{sx}: Mittelwert in kg stimmt nicht (body_mass_kg?)"
assert (table["male"] > table["female"]).all(), "Männchen sind in jeder Art im Mittel schwerer"
print(table.round(3))
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 4.5 #3

7 min

Wie vollständig ist das Geschlecht erfasst? Schreiben Sie `sex_coverage(df)`: eine Tabelle mit den Arten in den Zeilen und zwei Spalten `size` (Tiere je Art) und `count` (davon mit Geschlecht). Wenn Sie die Zeilen ohne Geschlecht behalten haben, zeigt sie die Lücken je Art; wenn Sie sie entfernt haben, sind beide Spalten gleich, und das ist dann die Aussage.

Hinweis: `pd.DataFrame({"size": g.size(), "count": g.count()})` mit `g = df.groupby("species")["sex"]`, wie die Zelle mit `size` und `count` im Termin.

→ Lösung im Anhang

```
def sex_coverage(df):
    # TODO: Ihre Lösung hier
    ...
```

```
   # Prüfzelle – einfach ausführen
   table = sex_coverage(clean)
   assert table is not None, "sex_coverage gibt noch nichts zurück"
   assert list(table.columns) == ["size", "count"] and list(table.index) == ["Adelie", "Chinstrap", "Gentoo"],
"Spalten size und count, Arten in den Zeilen"
   for sp in table.index:
       mask = clean["species"] == sp
       assert table.loc[sp, "size"] == mask.sum(), f"{sp}: size ist die Anzahl der Tiere"
       assert table.loc[sp, "count"] == (mask & clean["sex"].notna()).sum(), f"{sp}: count zählt nur Tiere mit
Geschlecht"
   assert (table["size"] - table["count"]).sum() == clean["sex"].isna().sum(), "die Differenz ist die Zahl der
Lücken"
   print(table)
   print("Alle Prüfungen bestanden ✓")
```

> 💭 **Kurz überlegen**
>
> Wer die Tiere ohne Geschlecht entfernt hat, sieht in `sex_coverage` zwei gleiche Spalten. Belegt das, dass das Geschlecht in der Studie vollständig erhoben wurde?

4.6 Analyseblatt

Setzt voraus: LE04, Kapitel LE 4, Abschnitt 4.6.

Das Analyseblatt der Forschungsgruppe: drei Koordinatensysteme nebeneinander. Links die Anzahl je Art als Balken, in der Mitte die Verteilung der Körpermasse als Histogramm mit festen Intervallen von 250 Gramm, rechts Schnabellänge gegen Schnabeltiefe als Streudiagramm mit einer Farbe je Art. Die drei Datenfunktionen sind diesmal vorgegeben, sie sind je eine Zeile wie im Termin; Ihre Arbeit ist das Zeichnen und danach das Lesen der Diagramme. Hinter dem Streudiagramm steht die Frage des Auftrags: Lassen sich die Arten an den Schnabelmaßen unterscheiden?

```
   # Vorgabe: die drei Datenfunktionen – einfach ausführen
   def species_counts(df):
       return df["species"].value_counts().sort_index()                # Anzahl je Art, alphabetisch

   def mass_values(df):
       return df["body_mass_g"]                                        # keine Lücken mehr, das hat der
Vertrag festgelegt

   def culmen_pairs(df):
       return df[["culmen_length_mm", "culmen_depth_mm", "species"]]   # die Paare bleiben zeilenweise
zusammen
① print(species_counts(clean).to_dict(), "|", len(mass_values(clean)), "Massen |", culmen_pairs(clean).shape)
```

① ein kurzer Blick auf das, was die drei liefern

```
{'Adelie': 151, 'Chinstrap': 68, 'Gentoo': 123} | 342 Massen | (342, 3)
```

✏️ Aufgabe 4.6 #1 — 12 min

Zeichnen Sie in das Gerüst: in `ax1` die Balken aus `species_counts(clean)`, in `ax2` das Histogramm von `mass_values(clean)` mit `bins=MASS_BINS`, in `ax3` das Streudiagramm Schnabellänge (x) gegen Schnabeltiefe (y) mit einem `scatter` je Art und `label`, dazu Legende. Alle drei mit Achsenbeschriftung und Titel, die Anzahl der Tiere im Titel des Histogramms.

Hinweis: Alles steht in den Demo-Zellen des Termins; `rates.index.astype(str)` wird hier `counts.index`, die Schleife `for species, group in culmen_pairs(clean).groupby("species"):` liefert je Art die Teiltabelle.

→ Lösung im Anhang

```
MASS_BINS = range(2500, 6750, 250)        # 16 Intervalle à 250 g
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(13, 3.6))
# TODO: Ihre Lösung hier
...
fig.tight_layout()
plt.show()
```

```
# Prüfzelle – einfach ausführen (dieselbe Prüfung wie im Termin, an Ihren Daten)
counts = species_counts(clean)
assert len(ax1.patches) == 3, f"drei Balken erwartet, nicht {len(ax1.patches)}"
assert [p.get_height() for p in ax1.patches] == counts.tolist(), "Balkenhöhen sollen die Anzahl je Art sein (alphabetisch)"
assert ax1.get_xlabel() and ax1.get_ylabel() and ax1.get_title(), "ax1: Achsen und Titel beschriften"
heights = [p.get_height() for p in ax2.patches]
expected = np.histogram(clean["body_mass_g"], bins=MASS_BINS)[0].tolist()
assert len(ax2.patches) == 16, f"16 Balken für die Intervalle 2500 bis 6500 erwartet, nicht {len(ax2.patches)}"
assert [round(p.get_x()) for p in ax2.patches] == list(MASS_BINS)[:-1], "die Balken sollen bei 2500, 2750, … beginnen (bins=MASS_BINS)"
assert heights == expected, f"Balkenhöhen {heights} passen nicht zu den Häufigkeiten {expected}"
assert str(len(clean)) in ax2.get_title(), "ax2: die Anzahl der Tiere gehört in den Titel"
assert ax2.get_xlabel() and ax2.get_ylabel(), "ax2: beide Achsen beschriften"
assert len(ax3.collections) == 3 and ax3.get_legend() is not None, "ax3: ein scatter je Art und eine Legende"
① def sorted_rows(a):
    a = np.asarray(a, dtype=float).round(4)
    return a[np.lexsort((a[:, 1], a[:, 0]))]
handles, labels = ax3.get_legend_handles_labels()
assert set(labels) == {"Adelie", "Chinstrap", "Gentoo"}, f"ax3: die Legende soll die drei Arten nennen, nicht {labels}"
for handle, lab in zip(handles, labels):
    drawn = np.asarray(handle.get_offsets(), dtype=float)
    wanted = clean.loc[clean["species"] == lab, ["culmen_length_mm", "culmen_depth_mm"]].to_numpy()
    assert len(drawn) == len(wanted), f"'{lab}': {len(wanted)} Punkte erwartet, gezeichnet {len(drawn)}"
    assert np.array_equal(sorted_rows(drawn), sorted_rows(wanted)), f"'{lab}': Punkte sollen (Länge, Tiefe) dieser Art sein; Achsen vertauscht?"
assert ax3.get_xlabel() and ax3.get_ylabel(), "ax3: beide Achsen beschriften"
print("Alle Prüfungen bestanden ✓")
```

① `get_legend_handles_labels()` liefert Punktmengen und label-Texte paarweise; je Art werden die Koordinaten verglichen

💭 Kurz überlegen

Welche Arten lassen sich an den Schnabelmaßen trennen, welche nicht?

💭 Kurz überlegen

Das Schnabelverhältnis `culmen_ratio` aus `add_features` fasst beide Maße in einer Zahl zusammen. Trennt diese eine Zahl die Arten besser als die zwei Maße im Streudiagramm? Schätzen Sie, dann rechnet die nächste Zelle nach.

```
print(feat.groupby("species")["culmen_ratio"].agg(["min", "max"]).round(2))
```

```
           min   max
species
Adelie     1.64  2.45
Chinstrap  2.35  3.26
Gentoo     2.57  3.61
```

4.7 Von der Tabelle zum Array

Setzt voraus: LE04, Kapitel LE 4, Abschnitt 4.7; LE03, Kapitel LE 3, Abschnitt 3.3.4, LE 3, Abschnitt 3.5 und LE 3, Abschnitt 3.5.2.

Ab hier rechnet NumPy. Ein Modell sieht keine Spaltennamen, sondern eine Matrix: eine Zeile je Tier, eine Spalte je Maß. Der Weg dorthin ist `to_numpy()`, und was danach kommt, kennen Sie aus LE03: Reduktionen entlang einer Achse, Broadcasting, Masken und der Abstand zwischen allen Paaren auf einmal. Drei Dinge rechnen Sie hier auf dem Array, die in LE05 die Pipeline übernimmt, dann aber hinter einem Aufruf versteckt: die Maße standardisieren, die Mittelwerte je Art, und als Zusatz der nächste Nachbar als erste Vorhersage der Art.

✏️ Aufgabe 4.7 #1 8 min

Schreiben Sie `to_arrays(df)`: `X_num` aus den vier Maßen in dieser Reihenfolge (`culmen_length_mm`, `culmen_depth_mm`, `flipper_length_mm`, `body_mass_g`) als Array mit `dtype="float32"`, `y` die Artnamen aus `species` als Array, Rückgabe als Tupel.

Hinweis: `df[cols].to_numpy(dtype="float32")` und `df["species"].to_numpy()`, wie im letzten Kapitel des Termins.

→ Lösung im Anhang

```
def to_arrays(df):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
result = to_arrays(clean)
assert result is not None, "to_arrays gibt noch nichts zurück (return vergessen?)"
X_num, y = result
assert X_num.shape == (len(clean), 4) and X_num.dtype == np.float32, f"X_num: Form ({len(clean)}, 4) und float32 erwartet, nicht {X_num.shape} {X_num.dtype}"
assert np.isnan(X_num).sum() == 0, "keine Lücken in den Maßen"
assert np.allclose(X_num[:, 0], clean["culmen_length_mm"].to_numpy(dtype="float32")), "Spalte 0 ist die Schnabellänge"
assert np.allclose(X_num[:, 3], clean["body_mass_g"].to_numpy(dtype="float32")), "Spalte 3 ist die Körpermasse"
assert y.shape == (len(clean),) and set(y) == {"Adelie", "Chinstrap", "Gentoo"}, "y: die Artnamen, Form (N,)"
print("X_num:", X_num.shape, X_num.dtype, "| y:", y.shape, "mit", len(set(y)), "Arten")
print("Alle Prüfungen bestanden ✓")
```

Die vier Spalten haben vier verschiedene Maßstäbe. Die nächste Zelle zeigt, wie verschieden: `axis=0` fasst entlang der Zeilen zusammen, also je Spalte.

```
X_num, y = to_arrays(clean)
print("Mittelwert je Spalte:   ", X_num.mean(axis=0).round(1))
print("Streuung (std) je Spalte:", X_num.std(axis=0).round(1))
```

```
Mittelwert je Spalte:    [  43.9   17.2  200.9 4201.8]
Streuung (std) je Spalte: [  5.5   2.   14.  800.8]
```

Die Körpermasse streut um 800, die Schnabeltiefe um 2. Wer in diesem Array den Abstand zwischen zwei Tieren berechnet, misst fast nur die Massendifferenz, die drei anderen Maße gehen unter. Deshalb werden die Spalten vor jedem Abstand oder Modell standardisiert: je Spalte den Mittelwert abziehen und durch die Streuung teilen. Danach hat jede Spalte den Mittelwert 0 und die Streuung 1. In NumPy ist das eine Zeile, weil Broadcasting das Array der Form `(N, 4)` und den Vektor der Form `(4,)` zeilenweise zusammenbringt.

> 💭 **Kurz überlegen**
>
> Bevor Sie rechnen: `X_num` hat die Form `(342, 4)`. Welche Form muss `mean` haben, wenn jede Messgröße einzeln standardisiert werden soll, und welches `axis` liefert sie? Begründen Sie, dann prüfen Sie mit der Zelle danach.

✏️ Aufgabe 4.7 #2 — 7 min

Schreiben Sie `standardize(X)`: Sie berechnet `mean` und `std` je Spalte (`axis=0`), bildet `Z = (X - mean) / std` und gibt das Tripel `Z, mean, std` zurück. Die Eingabe bleibt unverändert; `mean` und `std` werden zurückgegeben, weil ein Modell neue Tiere später mit denselben Werten umrechnen muss, nicht mit den Werten der neuen Tiere.

Hinweis: `X.mean(axis=0)` hat die Form `(4,)`, `X - X.mean(axis=0)` die Form `(N, 4)`. Kein `-=` auf `X`, das würde die Eingabe überschreiben.

→ Lösung im Anhang

```
def standardize(X):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
before = X_num.copy()
result = standardize(X_num)
assert result is not None and len(result) == 3, "standardize soll Z, mean und std zurückgeben"
Z, mean, std = result
assert np.array_equal(X_num, before), "die Eingabe wurde verändert (-= statt - ?)"
assert Z.shape == X_num.shape and mean.shape == (4,) and std.shape == (4,), f"Formen: Z {Z.shape}, mean
{mean.shape}, std {std.shape}"
assert np.allclose(Z.mean(axis=0), 0, atol=1e-3), f"jede Spalte von Z soll Mittelwert 0 haben, hat
{Z.mean(axis=0).round(4)}"
assert np.allclose(Z.std(axis=0), 1, atol=1e-3), f"jede Spalte von Z soll Streuung 1 haben, hat
{Z.std(axis=0).round(4)}"
ref = (clean["body_mass_g"].to_numpy() - clean["body_mass_g"].mean()) / clean["body_mass_g"].std(ddof=0)
# zweiter Weg, in pandas
assert np.allclose(Z[:, 3], ref, atol=1e-3), "Spalte 3 von Z soll die standardisierte Körpermasse sein"
print("mean:", mean.round(1), "| std:", std.round(1))
print("Streuung nach dem Standardisieren:", Z.std(axis=0).round(3))
print("Alle Prüfungen bestanden ✓")
```

> 💭 **Kurz überlegen**
>
> In LE03 haben Sie mit `row_l2_normalize` jeden Zeilenvektor normiert. Hier standardisieren Sie jede Spalte. Warum die andere Achse?

Die dritte Rechnung ist eine, die Sie in pandas schon gemacht haben: der Mittelwert je Art. In NumPy gibt es kein `groupby`. Stattdessen wählt eine Maske `y == "Gentoo"` die Zeilen einer Art, und `mean(axis=0)` fasst sie zusammen; drei Masken, drei Zeilen, `np.stack` legt sie übereinander. Die Prüfzelle vergleicht das Ergebnis mit `groupby`: zwei Werkzeuge, dieselben Zahlen.

✏️ Aufgabe 4.7 #3 — 7 min

Schreiben Sie `species_means(X, y)`: ein Array der Form `(3, 4)` mit dem Mittelwert je Maß und Art, die Arten in der Reihenfolge von `np.unique(y)` (alphabetisch).

Hinweis: `np.unique(y)` liefert die Arten sortiert. Je Art eine Maske `y == sp`, damit die Zeilen auswählen, `mean(axis=0)` wie eben; `np.stack` legt die drei Ergebnisvektoren zu einem Array `(3, 4)` übereinander.

→ Lösung im Anhang

```
def species_means(X, y):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
M = species_means(X_num, y)
assert M is not None, "species_means gibt noch nichts zurück"
assert M.shape == (3, 4), f"Form (3, 4) erwartet, nicht {M.shape}"
MEASURES = ["culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"]
by_pandas = clean.groupby("species")[MEASURES].mean().to_numpy()        # derselbe Mittelwert, in pandas
assert np.allclose(M, by_pandas, rtol=1e-4), "die Mittelwerte sollen mit groupby übereinstimmen
(Reihenfolge Adelie, Chinstrap, Gentoo?)"
print(pd.DataFrame(M, index=np.unique(y), columns=MEASURES).round(1))
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 4.7 #4

Zusatzaufgabe, 12 min

Die erste Vorhersage der Art, ohne Modell: Für jedes Tier suchen Sie das Tier mit dem kleinsten Abstand im standardisierten Array und sagen dessen Art voraus. Schreiben Sie `nearest_species(Z, y)`: Sie gibt `pred` zurück, die Art des nächsten Nachbarn für jede Zeile, Form `(N,)`. Ein Tier darf nicht sein eigener Nachbar sein. Wie oft stimmt die Vorhersage? Und welche Arten werden verwechselt?

Fachlicher Hinweis (erst das): Vier Teilschritte: alle Abstände zwischen allen Paaren auf einmal berechnen, wie in LE03 bei Compare; den Abstand jedes Tiers zu sich selbst ausschließen; je Zeile die Position des kleinsten Abstands finden; mit dieser Position die Art aus `y` holen.

Technischer Hinweis (wenn ein Schritt hakt): `diff = Z[:, None, :] - Z[None, :, :]` hat die Form `(N, N, 4)`, `dist = np.sqrt((diff ** 2).sum(axis=2))` die Form `(N, N)`. `np.fill_diagonal(dist, np.inf)`, `dist.argmin(axis=1)`, `y[...]`.

→ Lösung im Anhang

```
def nearest_species(Z, y):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
pred = nearest_species(Z, y)
assert pred is not None, "nearest_species gibt noch nichts zurück"
assert pred.shape == y.shape and set(pred) <= {"Adelie", "Chinstrap", "Gentoo"}, "pred: eine Art je Zeile"
def nearest_other(i):                                                    # Gegenprobe Tier für Tier, ohne
Broadcasting über alle Paare
    d = np.sqrt(((Z - Z[i]) ** 2).sum(axis=1))
    d[i] = np.inf
    return y[d.argmin()]
ref = np.array([nearest_other(i) for i in range(len(Z))])
assert (pred == ref).mean() >= 0.995, f"pred soll je Zeile die Art des nächsten anderen Tiers sein; stimmt
nur in {(pred == ref).mean():.1%} der Zeilen"
accuracy = (pred == y).mean()
assert accuracy > 0.85, f"mit standardisierten Maßen stimmen weit über 85 % der Vorhersagen, hier
{accuracy:.1%}"
raw_pred = nearest_species(X_num.astype("float64"), y)
print(f"Treffer standardisiert: {accuracy:.1%} | Treffer auf den Rohmaßen: {(raw_pred == y).mean():.1%}")
print(pd.crosstab(pd.Series(y, name="Art"), pd.Series(pred, name="Vorhersage")))
print("Alle Prüfungen bestanden ✓")
```

> 💭 **Kurz überlegen**
>
> Die Prüfzelle rechnet die Vorhersage auch auf den Rohmaßen. Warum ist sie dort schlechter, und warum braucht die Standardisierung `mean` und `std` als Rückgabe?

4.8 Übergabe

Setzt voraus: Grundkurs Teil 3, Kapitel LE 3, Abschnitt 3.1.

Die Übergabe an die nächste Lerneinheit ist die bereinigte Tabelle als CSV-Datei, nicht das Array: Die Tabelle enthält mit `sex` eine Kategorie mit Lücken und mit `island` eine zweite Kategorie, und genau damit beschäftigt sich die Pipeline in LE05 (Lücken füllen, Kategorien kodieren, Maße skalieren, gelernt nur auf den Trainingsdaten). Das Array und die Standardisierung aus dem letzten Kapitel baut sie aus der Tabelle neu, dann mit scikit-learn.

✏️ Aufgabe 4.8 #1 7 min

Speichern Sie `clean` als `out/penguins_clean.csv` ohne Index-Spalte, lesen Sie die Datei als `back` zurück und vergleichen Sie beide Tabellen mit `pd.testing.assert_frame_equal(back, clean, check_dtype=False)`. Die Funktion bricht mit einer Meldung ab, wenn sich die Tabellen unterscheiden, und ist still, wenn alles passt.

Zusatzaufgabe (JSON): Speichern Sie die Kennzahlen Ihrer Tabelle als `out/penguins_summary.json`: ein Dict mit der Anzahl je Art und der mittleren Körpermasse je Art. `json.dump` nimmt NumPy-Ganzzahlen und `float32` nicht an (`TypeError`), `float64` schon; wandeln Sie die Werte vorher mit `int()` und `float()` um. Lesen Sie die Datei zurück und vergleichen Sie.

Zusatzaufgabe (Modul): Schreiben Sie `RENAME`, `load_penguins`, `clean_penguins` und `add_features` mit `%%writefile penguin_tools.py` in ein Modul und führen Sie die Pipeline aus dem Modul in drei Zeilen aus; `importlib.reload`, falls Sie die Datei ändern.

Hinweis: `check_dtype=False`, weil `equals` hier am Datentyp der gekürzten Artspalte scheitern kann (`object` gegen `str`), obwohl jeder Wert gleich ist. Was zählt, sind die Werte.

→ *Lösung im Anhang*

```
# TODO: Ihre Lösung hier
...
```

```
# Prüfzelle – einfach ausführen (liest die Datei selbst)
assert os.path.exists("out/penguins_clean.csv"), "out/penguins_clean.csv fehlt"
disk = pd.read_csv("out/penguins_clean.csv")
assert disk.shape == clean.shape, f"die Datei hat {disk.shape}, clean hat {clean.shape} (index=False vergessen?)"
pd.testing.assert_frame_equal(disk, clean, check_dtype=False)
assert disk["sex"].isna().sum() == clean["sex"].isna().sum(), "Lücken in sex bleiben beim Speichern erhalten"
print("Alle Prüfungen bestanden ✓")
```

```
# Lösung der Zusatzaufgabe (JSON) in einer eigenen Zelle
# TODO: Ihre Lösung hier
...
```

```
%%writefile penguin_tools.py
"""Werkzeuge der Projektaufgabe: laden, bereinigen, Merkmale ableiten."""
import pandas as pd

RENAME = {
    "studyName": "study", "Sample Number": "sample_number", "Species": "species", "Region": "region",
    "Island": "island", "Stage": "stage", "Individual ID": "individual_id",
    "Clutch Completion": "clutch_completion", "Date Egg": "date_egg",
    "Culmen Length (mm)": "culmen_length_mm", "Culmen Depth (mm)": "culmen_depth_mm",
    "Flipper Length (mm)": "flipper_length_mm", "Body Mass (g)": "body_mass_g", "Sex": "sex",
    "Delta 15 N (o/oo)": "delta_15n", "Delta 13 C (o/oo)": "delta_13c", "Comments": "comments",
}

# TODO: Ihre Lösung hier
...
```

```
# Lösung der Zusatzaufgabe (Modul): die Pipeline aus dem Modul
# TODO: Ihre Lösung hier
...
```

✏️ Aufgabe 4.8 #2 — 10 min

Verständnisnachweis. Beantworten Sie die drei Leitfragen vom Anfang in der Markdown-Zelle darunter, je drei bis fünf Sätze, mit Zahlen aus Ihrem Notebook. Die Antworten besprechen wir zu Beginn von LE05.

1. Welche Bereinigungsentscheidungen haben Sie getroffen, und welche Folge hat jede für die Auswertungen danach?
2. Welche Aussage trägt Ihr Analyseblatt, und welche Aussage wäre zu weitgehend?
3. Warum standardisieren Sie spaltenweise, und warum geben Sie `mean` und `std` zurück, statt sie wegzuwerfen?

Hinweis: Stichworte genügen nicht ganz, aber ein Aufsatz ist nicht gemeint. Zu 1 helfen `CONTRACT["decisions"]` und `sex_coverage`, zu 2 die zwei Denkfragen am Analyseblatt, zu 3 die zwei Denkfragen im Array-Kapitel.

→ *Lösung im Anhang*

Ihre Antworten:

1. …
2. …
3. …

Überblick: Die Entscheidungen dieser Aufgabe

Befund in der Rohtabelle	Entscheidung	Stelle im Code
Spaltennamen mit Leerzeichen und Einheiten	kurze Namen, Einheit bleibt im Namen	`RENAME`, `rename(columns=…)`
Kennungen, konstante Spalten, Kommentare, Isotope	nicht in die bereinigte Tabelle	Spaltenauswahl in `clean_penguins`
Zwei Tiere ohne Maße	entfallen	`dropna(subset=[Maße])`
Elf Tiere ohne Geschlecht	Ihre Entscheidung, begründet	`CONTRACT["decisions"]`, `nullable` bei `sex`
Art als Langtext, Großschreibung uneinheitlich	Kurzname aus dem ersten Wort	`.str.split(" ").str[0]`
`Sex` in Großbuchstaben, `Clutch Completion` als Ja/Nein	`lower()`, `bool` nach Prüfung	`.str.lower()`, `== "Yes"`
Flossenlänge ganzzahlig, aber `float64`	`int64`, sobald keine Lücken mehr	`.astype("int64")`
Was der Auftrag braucht und die Datei nicht hat	abgeleitete Spalten	`add_features`
Vier Maßstäbe in einem Array	je Spalte standardisieren, `mean` und `std` aufheben	`standardize`, `axis=0` und Broadcasting

Diese Tabelle ist das Protokoll der Bereinigung. Für einen eigenen Datensatz ist sie die Vorlage: Befund, Entscheidung, Stelle im Code.

➡️ Nächste Schritte

- Wenn eine Prüfzelle rot bleibt, lesen Sie die Meldung: Sie nennt Spalte und Befund. Die Lösungen zum Aufklappen und die Anschlusslösungen zeigen eine Fassung, die den Vertrag erfüllt; Ihre darf anders aussehen, solange die Prüfzellen grün sind.
- Heben Sie `out/penguins_clean.csv` auf. LE05 baut darauf die erste Pipeline mit scikit-learn: Lücken in `sex` füllen, `island` und `sex` kodieren, die Maße skalieren, dann ein Modell, das die Art vorhersagt, und die Prüfung, ob die Vorhersage aus dem Streudiagramm stimmt.
- Die Referenzkarte pandas fasst alle Werkzeuge dieser Aufgabe auf zwei Seiten zusammen, Matplotlib darauf als Vorgriff.

→ **Ausblick:** Grundkurs Teil 4 (Klassen, Signaturen, Stil) kommt mit LE05 und bereitet den Einstieg in PyTorch vor.

Quellen und Referenzen

- Datengrundlage: K. B. Gorman, T. D. Williams, W. R. Fraser: *Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus Pygoscelis)*, PLoS ONE 9(3), 2014, e90081. Rohdaten aus A. M. Horst, A. P. Hill, K. B. Gorman: palmerpenguins, R-Paket Version 0.1.0, 2020, allisonhorst.github.io/palmerpenguins, Lizenz CC0. Die Datei `penguins_raw.csv` ist unverändert, das Datenblatt `DATENBLATT_penguins.md` beschreibt die Spalten.
- W. McKinney: *Python for Data Analysis*, 3. Auflage, Kapitel 7 (Datenbereinigung) und 10 (Gruppieren), wesmckinney.com/book.
- pandas-Dokumentation: Working with missing data, Group by.
- Python-Tutorial, Kapitel 7.2 Dateien und 8 Fehler und Ausnahmen.

Dieses Material wurde mit Unterstützung von KI-Werkzeugen entwickelt (Text und Code: Claude und ChatGPT). Alle Inhalte sind von Prof. Dr. Stephan Kurpjuweit konzipiert, geprüft und verantwortet.

Anhang: Lösungen

4.1 Der Datensatz

✏️ Aufgabe 4.1 #1 5 min

Füllen Sie das Dict `roles` aus: ein Eintrag je Spalte der Rohtabelle, der Wert ist eine der sieben Rollen aus der Übersicht als Text. Die Prüfzelle prüft, ob jede Spalte eine gültige Rolle hat, und rechnet bei den konstanten Spalten nach. Bei `studyName` sind `id` und `date` beide richtig, denn der Name enthält die Saison.

```
roles = {
    "studyName": "id", "Sample Number": "id", "Species": "target", "Region": "constant",
    "Island": "category", "Stage": "constant", "Individual ID": "id", "Clutch Completion": "category",
    "Date Egg": "date", "Culmen Length (mm)": "measure", "Culmen Depth (mm)": "measure",
    "Flipper Length (mm)": "measure", "Body Mass (g)": "measure", "Sex": "category",
    "Delta 15 N (o/oo)": "measure", "Delta 13 C (o/oo)": "measure", "Comments": "comment",
}
```

↑ zur Aufgabenstellung

💭 Kurz überlegen 4.1 #1

`Individual ID` hat 190 verschiedene Werte für 344 Zeilen. Ist es dann eine Kennung?

> Eine Kennung der Erhebung, nicht der Zeile: Die Nummern wurden je Saison vergeben, `N1A1` steht einmal für ein Adelie-Männchen und einmal für ein Gentoo-Weibchen. Erst `studyName` zusammen mit `Individual ID` kennzeichnet eine Zeile eindeutig. Für das Modell ist beides ohne Bedeutung, eine Nummer sagt nichts über das Tier. Die nächste Zelle rechnet es nach.

4.2 Laden mit Fehlervertrag

✏️ Aufgabe 4.2 #1 15 min

Schreiben Sie `load_penguins(path)` nach dem Vertrag. Die Prüfzelle prüft den Normalfall, eine kleine zweite Datei mit drei Zeilen und beide Fehlerfälle.

```
def load_penguins(path):
    """Liest die Rohtabelle der Palmer-Pinguine und benennt die Spalten um."""
    df = pd.read_csv(path, encoding="utf-8")
    missing = set(RENAME) - set(df.columns)
    if missing:
        raise ValueError(f"missing columns in {path}: {sorted(missing)}")
    return df.rename(columns=RENAME)
```

↑ zur Aufgabenstellung

4.3 Vertrag und Bereinigung

💭 Kurz überlegen 4.3 #1

Welche Lücken machen eine Zeile für den Auftrag unbrauchbar, und welche nicht?

> Unbrauchbar ist eine Zeile ohne Maße: Für die Auswertung der Körpermaße und für ein Modell, das die Art aus den Maßen bestimmt, trägt sie nichts bei. Das betrifft zwei Tiere, bei beiden steht im Kommentar `Adult not sampled.`. Alle anderen Lücken sitzen in Spalten, die wir gar nicht brauchen (Kommentare, Isotope), oder in einer Spalte, die wir brauchen können, aber nicht müssen: das Geschlecht fehlt bei elf Tieren, deren Maße vollständig sind. Diese Zeilen zu entfernen ist eine Entscheidung, keine Notwendigkeit. Deshalb wird `dropna` nur mit `subset=` verwendet, und die Spalten, die wir nicht brauchen, fliegen raus, nicht die Zeilen.

✏️ Aufgabe 4.3 #1 — 10 min

Der Vertrag ist ein Dict `CONTRACT`. Die sechs festgelegten Spalten sind schon eingetragen: je Spalte ein Eintrag mit `kind` (`"text"`, `"bool"`, `"int"` oder `"float"`) und entweder `values` (Menge der zulässigen Werte) oder `range` (Tupel aus Minimum und Maximum, das die Werte einschließen darf); `"nullable": True` steht dort, wo Lücken erlaubt sind, sonst entfällt der Schlüssel. Lesen Sie die sechs Zeilen, dann treffen Sie Ihre zwei Entscheidungen und ergänzen `sex` (mit oder ohne `nullable`) und `clutch_completion` (als `bool`, als Text oder gar nicht). Dazu unter `rows` in einem Satz, welche Zeilen entfallen, und unter `decisions` je Entscheidungsfeld die drei Teile Entscheidung, Begründung, Folge. Die Lösung zeigt eine Fassung, die die neun Zeilen ohne Geschlecht behält; die Alternative ist ebenso richtig.

```
CONTRACT = {
    "columns": {
        "species": {"kind": "text", "values": {"Adelie", "Chinstrap", "Gentoo"}},   # fest
        "island": {"kind": "text", "values": {"Biscoe", "Dream", "Torgersen"}},   # fest
        "culmen_length_mm": {"kind": "float", "range": (30.0, 65.0)},   # fest
        "culmen_depth_mm": {"kind": "float", "range": (12.0, 25.0)},    # fest
        "flipper_length_mm": {"kind": "int", "range": (170, 240)},      # fest
        "body_mass_g": {"kind": "float", "range": (2500.0, 6500.0)},    # fest
        # Ihre Entscheidungen: sex (nullable?) und clutch_completion (bool, Text oder weglassen)
        "sex": {"kind": "text", "values": {"female", "male"}, "nullable": True},
        "clutch_completion": {"kind": "bool", "values": {True, False}},
    },
    "rows": "...",                      # ein Satz: welche Zeilen entfallen
    "decisions": {                      # je Feld: Entscheidung. Begründung. Folge für die Auswertung.
        "Zeilen ohne Geschlecht": "Bleiben. Die Maße sind vollständig und bleiben verfügbar. Vergleiche
nach Geschlecht verwenden neun Tiere weniger.",
        "clutch_completion": "Bleibt, als bool. Zwei Werte Ja/Nein, vorher geprüft. Auswertungen können
danach filtern.",
    },
}
CONTRACT["rows"] = "Die zwei Zeilen ohne Maße entfallen, sonst keine."
```

↑ zur Aufgabenstellung

✏️ Aufgabe 4.3 #2 30 min

Schreiben Sie `clean_penguins(df)`: Sie bekommt die Tabelle aus `load_penguins` und gibt eine neue Tabelle zurück, die `CONTRACT` erfüllt und die Kursregeln einhält. Die Prüfzelle prüft zuerst Ihren Vertrag und danach die Regeln, die für jede Entscheidung gelten.

Zusatzaufgabe: Leiten Sie vor der Spaltenauswahl das Jahr ab (`study` ist `PAL0708` für die Saison 2007/08: `df["study"].str[3:5].astype(int) + 2000`) und nehmen Sie es als Spalte `year` in eine zweite Fassung von Funktion und Vertrag auf. Zählen Sie die Tiere je Jahr.

```
def clean_penguins(df):
    """Bereinigt die Rohtabelle nach CONTRACT und gibt eine neue Tabelle zurück."""
    out = df[["species", "island", "sex", "clutch_completion",
              "culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"]]
    out = out.dropna(subset=["culmen_length_mm", "culmen_depth_mm",
                             "flipper_length_mm", "body_mass_g"])              # die zwei Tiere ohne Maße
    out["species"] = out["species"].str.split(" ").str[0]            # "Adelie Penguin (...)" -> "Adelie"
    out["sex"] = out["sex"].str.lower()
    assert set(out["clutch_completion"]) == {"Yes", "No"}, "unerwartete Werte in clutch_completion"
    out["clutch_completion"] = out["clutch_completion"] == "Yes"
    out["flipper_length_mm"] = out["flipper_length_mm"].astype("int64")       # ganzzahlig, keine Lücken
mehr
    return out.drop_duplicates().reset_index(drop=True)
```

```
# Lösung der Zusatzaufgabe (Jahr) in einer eigenen Zelle: zweite Fassung von Funktion und Vertrag
def clean_penguins_year(df):
    out = df.assign(year=df["study"].str[3:5].astype(int) + 2000)     # "PAL0708" -> 2007, vor der
Spaltenauswahl
    out = out[["year"] + list(clean.columns)]
    out = out.dropna(subset=["culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"])
    out["species"] = out["species"].str.split(" ").str[0]
    out["sex"] = out["sex"].str.lower()
    out["clutch_completion"] = out["clutch_completion"] == "Yes"
    out["flipper_length_mm"] = out["flipper_length_mm"].astype("int64")
    return out.drop_duplicates().reset_index(drop=True)

CONTRACT_YEAR = {"columns": {"year": {"kind": "int", "values": {2007, 2008, 2009}},
**CONTRACT["columns"]},
                 "rows": CONTRACT["rows"], "decisions": {**CONTRACT["decisions"], "Jahr": "aufgenommen,
aus study"}}
clean_year = clean_penguins_year(load_penguins("data/penguins_raw.csv"))
check_contract(clean_year, CONTRACT_YEAR)
print(clean_year["year"].value_counts().sort_index().to_dict())
```

↑ zur Aufgabenstellung

4.4 Merkmale ableiten

✏️ Aufgabe 4.4 #1 8 min

Schreiben Sie `add_features(df)`: Sie gibt eine neue Tabelle mit den Spalten `body_mass_kg` (Masse in Kilogramm) und `culmen_ratio` (Länge durch Tiefe) zusätzlich zu allen vorhandenen zurück. Die Eingabe bleibt unverändert.

```python
def add_features(df):
    out = df.copy()
    out["body_mass_kg"] = out["body_mass_g"] / 1000
    out["culmen_ratio"] = out["culmen_length_mm"] / out["culmen_depth_mm"]
    return out
```

↑ zur Aufgabenstellung

4.5 Fragen an die Daten

💭 Kurz überlegen 4.5 #1

Bevor Sie die Tabelle bauen: Welche Form hat sie, und wie viele ihrer Einträge erwarten Sie als Null?

> Drei Arten mal drei Inseln, Form `(3, 3)`. Aus dem Datenblatt: Adelie wurde auf allen drei Inseln beobachtet, Chinstrap nur auf Dream, Gentoo nur auf Biscoe. Das ergibt vier Nullen. Vergleichen Sie nach der Prüfzelle mit Ihrer Vorhersage; wer etwas anderes erwartet hat, findet in der Tabelle den Grund.

✏️ Aufgabe 4.5 #1 8 min

Welche Arten wurden in diesem Datensatz auf welchen Inseln beobachtet? Schreiben Sie `counts_by_island(df)`: eine Tabelle mit den Arten in den Zeilen, den Inseln in den Spalten und der Anzahl der Tiere als Werte, Nullen statt Lücken.

```python
def counts_by_island(df):
    return pd.crosstab(df["species"], df["island"])
```

↑ zur Aufgabenstellung

✏️ Aufgabe 4.5 #2 10 min

Wie schwer sind die vermessenen Tiere je Art und Geschlecht, in Kilogramm? Schreiben Sie `mass_by_species_sex(df)`: mittlere Körpermasse aus der Spalte `body_mass_kg`, Arten in den Zeilen, Geschlechter in den Spalten. Die Funktion bekommt `feat` aus dem letzten Kapitel. Tiere ohne Geschlecht fallen dabei heraus, ohne dass Sie etwas tun müssen: `groupby` lässt fehlende Schlüssel weg.

```python
def mass_by_species_sex(df):
    return df.groupby(["species", "sex"])["body_mass_kg"].mean().unstack()
```

↑ zur Aufgabenstellung

✏️ Aufgabe 4.5 #3 7 min

Wie vollständig ist das Geschlecht erfasst? Schreiben Sie `sex_coverage(df)`: eine Tabelle mit den Arten in den Zeilen und zwei Spalten `size` (Tiere je Art) und `count` (davon mit Geschlecht). Wenn Sie die Zeilen ohne Geschlecht behalten haben, zeigt sie die Lücken je Art; wenn Sie sie entfernt haben, sind beide Spalten gleich, und das ist dann die Aussage.

```python
def sex_coverage(df):
    g = df.groupby("species")["sex"]
    return pd.DataFrame({"size": g.size(), "count": g.count()})
```

↑ zur Aufgabenstellung

💭 Kurz überlegen 4.5 #2

Wer die Tiere ohne Geschlecht entfernt hat, sieht in `sex_coverage` zwei gleiche Spalten. Belegt das, dass das Geschlecht in der Studie vollständig erhoben wurde?

> Nein. Die Spalten sind gleich, weil die Tabelle nur noch Tiere mit Geschlecht enthält; das ist die Folge der eigenen Auswahl, keine Eigenschaft der Erhebung. Die Rohdaten haben elf Lücken in `Sex`. Eine Kennzahl beschreibt die Tabelle, auf der sie berechnet wurde. Deshalb gehört zu jeder Auswertung der Satz, welche Zeilen vorher entfallen sind; genau dafür steht `rows` im Vertrag.

4.6 Analyseblatt

✏️ Aufgabe 4.6 #1 12 min

Zeichnen Sie in das Gerüst: in `ax1` die Balken aus `species_counts(clean)`, in `ax2` das Histogramm von `mass_values(clean)` mit `bins=MASS_BINS`, in `ax3` das Streudiagramm Schnabellänge (x) gegen Schnabeltiefe (y) mit einem `scatter` je Art und `label`, dazu Legende. Alle drei mit Achsenbeschriftung und Titel, die Anzahl der Tiere im Titel des Histogramms.

```python
MASS_BINS = range(2500, 6750, 250)        # 16 Intervalle à 250 g
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(13, 3.6))
counts = species_counts(clean)
ax1.bar(counts.index, counts.to_numpy())
ax1.set_xlabel("Art")
ax1.set_ylabel("Anzahl Tiere")
ax1.set_title("Tiere je Art")
masses = mass_values(clean)
ax2.hist(masses, bins=MASS_BINS, edgecolor="white")
ax2.set_xlabel("Körpermasse in g")
ax2.set_ylabel("Anzahl Tiere")
ax2.set_title(f"Körpermasse ({len(masses)} Tiere)")
for species, group in culmen_pairs(clean).groupby("species"):
    ax3.scatter(group["culmen_length_mm"], group["culmen_depth_mm"], s=12, alpha=0.7, label=species)
ax3.set_xlabel("Schnabellänge in mm")
ax3.set_ylabel("Schnabeltiefe in mm")
ax3.set_title("Schnabelmaße je Art")
ax3.legend()
fig.tight_layout()
plt.show()
```

↑ zur Aufgabenstellung

💭 Kurz überlegen 4.6 #1

Welche Arten lassen sich an den Schnabelmaßen trennen, welche nicht?

> Gentoo steht für sich: lange, flache Schnäbel, die Punktwolke liegt unten rechts ohne Überlappung. Adelie und Chinstrap haben ähnlich tiefe Schnäbel und überlappen in der Tiefe; in der Länge trennen sie sich, Adelie um 39 mm, Chinstrap um 49 mm. Ein Modell, das nur die Schnabeltiefe sieht, würde Adelie und Chinstrap verwechseln; mit beiden Maßen zusammen sind die drei Wolken fast getrennt. Das ist die Vorhersage, die LE05 nachprüft. Und was das Diagramm nicht trägt: eine Aussage über alle Pinguine dieser Arten. Es zeigt 342 Tiere von drei Inseln aus drei Saisons.

💭 Kurz überlegen 4.6 #2

Das Schnabelverhältnis `culmen_ratio` aus `add_features` fasst beide Maße in einer Zahl zusammen. Trennt diese eine Zahl die Arten besser als die zwei Maße im Streudiagramm? Schätzen Sie, dann rechnet die nächste Zelle nach.

> Schlechter. Die Bereiche überlappen: Adelie etwa 1.6 bis 2.5, Chinstrap 2.4 bis 3.3, Gentoo 2.6 bis 3.6. Chinstrap und Gentoo, die im Streudiagramm klar getrennt liegen, sind im Verhältnis kaum zu unterscheiden, weil ein langer flacher und ein mittellanger mitteltiefer Schnabel dasselbe Verhältnis haben können. Ein abgeleitetes Merkmal ist nicht automatisch besser als seine Bestandteile; es wirft Information weg, hier die absolute Größe. Für die Masse in Kilogramm gilt das nicht, sie ist nur eine Umskalierung.

4.7 Von der Tabelle zum Array

✏️ Aufgabe 4.7 #1 — 8 min

Schreiben Sie `to_arrays(df)`: `X_num` aus den vier Maßen in dieser Reihenfolge (`culmen_length_mm`, `culmen_depth_mm`, `flipper_length_mm`, `body_mass_g`) als Array mit `dtype="float32"`, `y` die Artnamen aus `species` als Array, Rückgabe als Tupel.

```python
def to_arrays(df):
    cols = ["culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"]
    X_num = df[cols].to_numpy(dtype="float32")
    y = df["species"].to_numpy()
    return X_num, y
```

↑ zur Aufgabenstellung

💭 Kurz überlegen 4.7 #1

Bevor Sie rechnen: `X_num` hat die Form `(342, 4)`. Welche Form muss `mean` haben, wenn jede Messgröße einzeln standardisiert werden soll, und welches `axis` liefert sie? Begründen Sie, dann prüfen Sie mit der Zelle danach.

> Form `(4,)`: ein Mittelwert je Spalte, denn jede Spalte ist eine Messgröße mit eigenem Maßstab. `axis=0` fasst entlang der Zeilen zusammen, also über alle Tiere, und lässt die vier Spalten stehen. `axis=1` ergäbe `(342,)`, einen Mittelwert je Tier aus Millimetern und Gramm, eine Zahl ohne Bedeutung. Beim Abziehen bringt Broadcasting `(342, 4)` und `(4,)` zusammen: Der Vektor wird an jede Zeile gelegt.

✏️ Aufgabe 4.7 #2 7 min

Schreiben Sie `standardize(X)`: Sie berechnet `mean` und `std` je Spalte (`axis=0`), bildet `Z = (X - mean) / std` und gibt das Tripel `Z, mean, std` zurück. Die Eingabe bleibt unverändert; `mean` und `std` werden zurückgegeben, weil ein Modell neue Tiere später mit denselben Werten umrechnen muss, nicht mit den Werten der neuen Tiere.

```python
def standardize(X):
    mean = X.mean(axis=0)
    std = X.std(axis=0)
    Z = (X - mean) / std
    return Z, mean, std
```

↑ zur Aufgabenstellung

💭 Kurz überlegen 4.7 #2

In LE03 haben Sie mit `row_l2_normalize` jeden Zeilenvektor normiert. Hier standardisieren Sie jede Spalte. Warum die andere Achse?

> Weil die Bedeutung der Daten eine andere ist. In LE03 war eine Zeile ein Embedding: ein Vektor, dessen Einträge gemeinsam eine Richtung beschreiben, und nur die Richtung zählte; deshalb wurde jede Zeile für sich auf Länge 1 gebracht. Hier ist eine Zeile ein Tier und eine Spalte eine Messgröße mit eigener Einheit; vergleichbar gemacht werden müssen die Messgrößen, nicht die Tiere. Die Achse folgt also aus der Frage, was die Einträge bedeuten, nicht aus einer Formel. Dieselbe Überlegung brauchen Sie bei jedem Tensor: erst klären, was Zeilen und Spalten sind, dann die Achse wählen.

✏️ Aufgabe 4.7 #3 7 min

Schreiben Sie `species_means(X, y)`: ein Array der Form `(3, 4)` mit dem Mittelwert je Maß und Art, die Arten in der Reihenfolge von `np.unique(y)` (alphabetisch).

```python
def species_means(X, y):
    return np.stack([X[y == sp].mean(axis=0) for sp in np.unique(y)])
```

↑ zur Aufgabenstellung

✏️ Aufgabe 4.7 #4 Zusatzaufgabe, 12 min

Die erste Vorhersage der Art, ohne Modell: Für jedes Tier suchen Sie das Tier mit dem kleinsten Abstand im standardisierten Array und sagen dessen Art voraus. Schreiben Sie `nearest_species(Z, y)`: Sie gibt `pred` zurück, die Art des nächsten Nachbarn für jede Zeile, Form `(N,)`. Ein Tier darf nicht sein eigener Nachbar sein. Wie oft stimmt die Vorhersage? Und welche Arten werden verwechselt?

```python
def nearest_species(Z, y):
    diff = Z[:, None, :] - Z[None, :, :]                # (N, N, 4): jedes Tier gegen jedes
    dist = np.sqrt((diff ** 2).sum(axis=2))             # (N, N)
    np.fill_diagonal(dist, np.inf)                      # das Tier selbst zählt nicht
    nearest = dist.argmin(axis=1)                       # (N,) Position des nächsten Nachbarn
    return y[nearest]
```

↑ zur Aufgabenstellung

💭 **Kurz überlegen 4.7 #3**

Die Prüfzelle rechnet die Vorhersage auch auf den Rohmaßen. Warum ist sie dort schlechter, und warum braucht die Standardisierung `mean` und `std` als Rückgabe?

> Auf den Rohmaßen ist der Abstand fast nur die Massendifferenz, und die Masse allein trennt Adelie und Chinstrap nicht (beide um 3.7 kg): rund 87 % Treffer. Standardisiert zählen alle vier Maße gleich, die Schnabellänge trennt die beiden, und die Trefferquote steigt auf rund 98 %. Die Verwechslungstabelle zeigt, was das Streudiagramm angekündigt hat: Gentoo wird fast nie verwechselt, die Fehler liegen zwischen Adelie und Chinstrap. `mean` und `std` müssen aufgehoben werden, weil ein neues Tier mit den Werten der Trainingsdaten umgerechnet werden muss; sonst läge es in einem anderen Koordinatensystem als die Tiere, mit denen es verglichen wird. Das ist die Regel, die LE05 mit `fit` auf den Trainingsdaten und `transform` auf allen Daten durchsetzt. Und noch eine Zahl: `dist` hat `N * N` Einträge, bei 342 Tieren sind das 117 000, bei 100 000 Tieren wären es zehn Milliarden. Der Trick mit allen Abständen auf einmal trägt für kleine Daten, für große braucht es andere Verfahren.

4.8 Übergabe

✏️ **Aufgabe 4.8 #1** 7 min

Speichern Sie `clean` als `out/penguins_clean.csv` ohne Index-Spalte, lesen Sie die Datei als `back` zurück und vergleichen Sie beide Tabellen mit `pd.testing.assert_frame_equal(back, clean, check_dtype=False)`. Die Funktion bricht mit einer Meldung ab, wenn sich die Tabellen unterscheiden, und ist still, wenn alles passt.

Zusatzaufgabe (JSON): Speichern Sie die Kennzahlen Ihrer Tabelle als `out/penguins_summary.json`: ein Dict mit der Anzahl je Art und der mittleren Körpermasse je Art. `json.dump` nimmt NumPy-Ganzzahlen und `float32` nicht an (`TypeError`), `float64` schon; wandeln Sie die Werte vorher mit `int()` und `float()` um. Lesen Sie die Datei zurück und vergleichen Sie.

Zusatzaufgabe (Modul): Schreiben Sie `RENAME`, `load_penguins`, `clean_penguins` und `add_features` mit `%%writefile penguin_tools.py` in ein Modul und führen Sie die Pipeline aus dem Modul in drei Zeilen aus; `importlib.reload`, falls Sie die Datei ändern.

```
clean.to_csv("out/penguins_clean.csv", index=False)
back = pd.read_csv("out/penguins_clean.csv")
pd.testing.assert_frame_equal(back, clean, check_dtype=False)
print("gespeichert und zurückgelesen:", back.shape)
```

```
# Lösung der Zusatzaufgabe (JSON) in einer eigenen Zelle
counts = species_counts(clean)
means = clean.groupby("species")["body_mass_g"].mean()
summary = {"n_rows": int(len(clean)),
           "count_by_species": {sp: int(counts[sp]) for sp in counts.index},
           "mean_body_mass_g": {sp: round(float(means[sp]), 1) for sp in means.index}}
with open("out/penguins_summary.json", "w", encoding="utf-8") as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)
with open("out/penguins_summary.json", encoding="utf-8") as f:
    loaded = json.load(f)
assert loaded == summary, "zurückgelesen soll dasselbe Dict herauskommen"
print(loaded["count_by_species"], loaded["mean_body_mass_g"])
```

```
%%writefile penguin_tools.py
"""Werkzeuge der Projektaufgabe: laden, bereinigen, Merkmale ableiten."""
import pandas as pd

RENAME = {
```

```
    "studyName": "study", "Sample Number": "sample_number", "Species": "species", "Region": "region",
    "Island": "island", "Stage": "stage", "Individual ID": "individual_id",
    "Clutch Completion": "clutch_completion", "Date Egg": "date_egg",
    "Culmen Length (mm)": "culmen_length_mm", "Culmen Depth (mm)": "culmen_depth_mm",
    "Flipper Length (mm)": "flipper_length_mm", "Body Mass (g)": "body_mass_g", "Sex": "sex",
    "Delta 15 N (o/oo)": "delta_15n", "Delta 13 C (o/oo)": "delta_13c", "Comments": "comments",
}

def load_penguins(path):
    """Liest die Rohtabelle der Palmer-Pinguine und benennt die Spalten um."""
    df = pd.read_csv(path, encoding="utf-8")
    missing = set(RENAME) - set(df.columns)
    if missing:
        raise ValueError(f"missing columns in {path}: {sorted(missing)}")
    return df.rename(columns=RENAME)


def clean_penguins(df):
    """Bereinigt die Rohtabelle nach CONTRACT und gibt eine neue Tabelle zurück."""
    out = df[["species", "island", "sex", "clutch_completion",
              "culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"]]
    out = out.dropna(subset=["culmen_length_mm", "culmen_depth_mm",
                             "flipper_length_mm", "body_mass_g"])              # die zwei Tiere ohne Maße
    out["species"] = out["species"].str.split(" ").str[0]            # "Adelie Penguin (...)" -> "Adelie"
    out["sex"] = out["sex"].str.lower()
    assert set(out["clutch_completion"]) == {"Yes", "No"}, "unerwartete Werte in clutch_completion"
    out["clutch_completion"] = out["clutch_completion"] == "Yes"
    out["flipper_length_mm"] = out["flipper_length_mm"].astype("int64")       # ganzzahlig, keine Lücken
mehr
    return out.drop_duplicates().reset_index(drop=True)


def add_features(df):
    out = df.copy()
    out["body_mass_kg"] = out["body_mass_g"] / 1000
    out["culmen_ratio"] = out["culmen_length_mm"] / out["culmen_depth_mm"]
    return out
```

```
# Lösung der Zusatzaufgabe (Modul): die Pipeline aus dem Modul
import importlib, penguin_tools
importlib.reload(penguin_tools)
module_clean = penguin_tools.clean_penguins(penguin_tools.load_penguins("data/penguins_raw.csv"))
pd.testing.assert_frame_equal(module_clean, clean, check_dtype=False)
print(penguin_tools.__file__.split(os.sep)[-1], "liefert dieselbe Tabelle:", module_clean.shape)
```

↑ zur Aufgabenstellung

✏️ Aufgabe 4.8 #2 — 10 min

Verständnisnachweis. Beantworten Sie die drei Leitfragen vom Anfang in der Markdown-Zelle darunter, je drei bis fünf Sätze, mit Zahlen aus Ihrem Notebook. Die Antworten besprechen wir zu Beginn von LE05.

1. Welche Bereinigungsentscheidungen haben Sie getroffen, und welche Folge hat jede für die Auswertungen danach?
2. Welche Aussage trägt Ihr Analyseblatt, und welche Aussage wäre zu weitgehend?
3. Warum standardisieren Sie spaltenweise, und warum geben Sie `mean` und `std` zurück, statt sie wegzuwerfen?

↑ zur Aufgabenstellung